Do Active Units Reflect a Known State Space? Variational Autoencoders on Blocksworld

Klein Tahiraj

klein.tahiraj@studio.unibo.it

Project Work in Deep Learning (3 CFU), Prof. Andrea Asperti, Università di Bologna
November 2026

Abstract. Asperti suggested that, for a given architecture, there seems to be an internal dimension of data, which the active units of a variational autoencoder might reflect. We test this idea on Blocksworld, whose 866 legal states are all known, by training β-VAEs on its states and on two controls, and by comparing the number of active units with dimensions we fixed before training. At β = 1, five of our six Blocksworld settings keep a median of 15 or 16 of their 32 latents active, which matches none of these dimensions. The learning rate in one network, the encoding of the states and, in a single-seed pilot, the KL warm-up move this count. In a follow-up, ten independent bits give no active unit when each is stored once and ten when each is copied. Most active units of our reference configuration detect a block on another at one end of their range and the reverse at the other. In the main sweep the β-VAEs at β = 0.25 and 1 reconstruct every state from their posterior means, but their prior samples are often illegal.

1 Introduction

In this project we train variational autoencoders (VAEs) on the states of the planning domain Blocksworld [1], to see whether the latent units they keep active reflect a structure of the data that we know in advance. A VAE with more latents than it needs can leave some of them inactive, namely their posterior mean hardly changes with the input. Burda et al. found far fewer *active units* than latent dimensions, and adding dimensions did not raise their number [2]. Asperti, for whom this sparsity is a form of self-regularisation, suggested that "for a given architecture, there seems to exist an intrinsic internal dimension of data" [3, p. 1]. Asperti and Trentin later found that the count also depends on the learning rate, and warned against reading it as the dimension of the data manifold [4].

To see what the count reflects we need data whose structure is known before training. Disentanglement studies use synthetic images and their generative factors for this [5], [6], and we take Blocksworld instead, whose states are sets of binary facts tied by logical constraints, so that all the *legal states* can be listed. The theory that links active units to the dimension of the data assumes continuous data [7], [8], and our states are discrete, so for us the question is empirical. We organise the report around the four questions of the project proposal.

- **Q1. Can the "right number" be fixed in advance?** Not uniquely, since the states can be described by anything from five dependent block variables to all of their binary facts, so before training we fixed several possible dimensions (Section 3).
- **Q2. How many variables stay active, and what does the count depend on?** At β = 1, a median of about half of the 32 latents in five of our six Blocksworld settings, and the count moves with the learning rate in one network, the encoding and redundancy in the input (Sections 5.1–5.4).
- **Q3. Does each active variable correspond to one fact?** Not one to one (Section 5.5).
- **Q4. Are generated states legal?** Often not, mostly because a block is not in exactly one place or a derived fact contradicts the positions (Section 5.6).

2 Background

Active units. Following Burda et al. [2, p. 7], we call a latent unit *active* when the variance over the data of its posterior mean is above 0.01. Approximating the averaged KL term, and assuming zero mean, Asperti derived that it is smallest when this variance and the mean posterior variance add up to one (Eq. 13 of [3]), which Asperti and Trentin call the variance law [4, p. 199442].

The reconstruction–KL balance. The β-VAE [5] minimises

$$\mathcal{L}_\beta = D + \beta R,$$

where the distortion D is the reconstruction error, on our states the binary cross-entropy summed over the facts, and the rate R is the average Kullback–Leibler (KL) term [9]. Their sum $D + R$ is the negative evidence lower bound (ELBO). At evaluation we decode the posterior mean and threshold each output at 0.5, so a model can reconstruct every state exactly even with a positive distortion.

In an experiment of Asperti, a latent variable collapsed when its gain in reconstruction became smaller than its KL divergence [10, p. 6]. When the bits are independent, a decoder that ignores the code and outputs the frequency of each bit is already as good as any other, so at $\beta = 1$ a code with zero rate is optimal [11]. Other optima can keep information, though, and training does not have to end in collapse [9]. Copying a bit changes this, since a fair bit copied c times costs such a decoder $c \ln 2$ nats, while sending it perfectly through the code costs at least $\ln 2$ of rate. Copies can therefore make informative codes worth their cost, but this does not tell us how many units become active, or whether each takes one bit.

3 Data and possible dimensions (Q1)

3.1 Blocksworld states

We describe the Blocksworld states [1] of five blocks A–E with 36 binary facts, namely on_table(X), on(X, Y), holding(X), clear(X) and hand_empty. Each block is on the table, held, or on one of the other four, so the 30 *position facts* on_table, on and holding are a one-hot code of five six-valued *block variables*. The *derived facts* clear and hand_empty follow from the positions. In the state of Figure 1a, for example, A is on B, B is on C, C and D are on the table and E is held, so A and D are clear and hand_empty is false. There are 866 legal states, a number we checked in three different ways. We can also encode each state by its *binary index*, namely its number in some fixed order written in 10 bits.

Our states come from action sequences that we had generated earlier, split into training, development and test sequences, and we draw training states with their frequencies in the training split. Each split contains all the legal states, so our test scores measure how well a model fits states it has already seen.

Before training we computed several possible dimensions of these states (Table 1), none of which is the unique true one. The block variables are not independent, since only 866 of their 7,776 joint values are legal. The *affine dimension*, the dimension of the smallest flat subspace containing all legal states, is 25, as the six position facts of a block always sum to one and the derived facts add nothing.

3.2 Controls with known structure

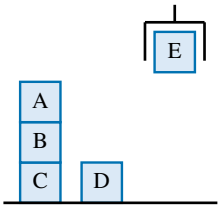
To tell these dimensions apart we built two 36-bit controls with known structure.

- *code10* has ten independent fair bits, each copied into three or four positions. Its entropy, its number of independent variables and its affine dimension are all ten.
- *prod5* is Blocksworld without the constraints between blocks, namely five independent six-valued variables in one-hot code, with six constant bits in place of the derived facts. It has the same block variables and affine dimension as Blocksworld, but a higher entropy.

Table 1. Possible dimensions of Blocksworld and of the two controls of Section 3.2, computed before training. The Blocksworld entropy uses the state frequencies over all sequences, and under the test frequencies it is 6.47 nats. The controls are uniform.

Possible dimension	Blocksworld	code10	prod5
$\log_2$ of the number of states n	9.758 bits	10 bits	12.925 bits
bits in a fixed-length state index, $\lceil \log_2 n \rceil$	10	10	13
entropy of the state distribution	9.346 bits	10 bits	12.925 bits
block or independent variables	5, dependent	10, independent	5, independent
affine dimension	25	10	25
binary inputs	36	36	36 (6 constant)

(a) A legal state and its true facts



on_table(C)
on_table(D)
on(A, B)
on(B, C)
clear(A)
clear(D)
holding(E)

(b) Decoded prior samples that break a constraint (2×256, $\beta = 1$)

A is clear, but D is on A

on_table(A) **clear(A)**
on_table(E) clear(B)
on(B, E) clear(D)
on(D, A)
holding(C)

B is not on the table, on a block or held

on_table(A) clear(A)
on_table(C) clear(D)
on(D, B) clear(E)
on(E, C) hand_empty
B has no place

the hand is neither empty nor holding a block

on_table(A) clear(C)
on_table(E) clear(D)
on(B, E)
on(C, A)
on(D, B)
hand_empty missing

Figure 1. (a) A legal state and its seven true facts. (b) Three prior samples of the reference configuration (Section 4.1, seed 1), decoded at 0.5, each the first we found that breaks only the constraint named above it. Facts in conflict and missing facts are in colour.

Before training we wrote down three hypotheses, each checked per network on the median counts at β = 1 and the lower learning rate. If the count follows the entropy of the states (*information hypothesis*), Blocksworld should stay within two units of code10, and prod5 more than two above Blocksworld. If it follows the affine dimension (*affine hypothesis*), Blocksworld should stay within two units of prod5 and more than two above code10, without having to reach 25. If it follows the number of block or independent variables (*factor-count hypothesis*), Blocksworld should stay within two units of prod5 and more than two below code10. prod5 differs from Blocksworld in three ways at once, so comparing the two cannot isolate one of them. We also train on 36 independent bits, each present once, where a code with zero rate is optimal.

4 Models and procedure

4.1 Models and training

Our main model is the β-VAE [5] with diagonal Gaussian posteriors, whose encoder and decoder are mirrored ReLU networks. We name them by the depth × width of their hidden layers, *1×64*, *2×256* and *2×1024*, and on the states we use 32 latents and a Bernoulli decoder. Our *reference configuration* is the 2×256 network on the 36 facts at β = 1 and learning rate (lr) $3{\cdot}10^{-4}$.

In Section 5.4 we train on the last-layer activations of five small transformers that we had trained earlier on Blocksworld sequences, named by their width T64, T128a–c (three seeds) and T256. There we use 64 latents and a Gaussian decoder whose variance γ is fixed at 1 or learned [7].

We compare the β-VAE with four other models.

- A plain autoencoder, with no KL term, checks the active-unit metric.
- Since each legal state is a constant vector plus one vector per block position, we train a top-k sparse autoencoder (SAE) [12] with 64 units and $k \in \{5, 10\}$.
- The β-TCVAE [13] penalises the dependence between latents, with weight $\beta_{\mathrm{TC}} \in \{2, 6\}$.
- The categorical VAE has 16 six-valued latents, like the block variables, and is trained on Gumbel-softmax samples [14].

All models use Adam [15] for 60,000 steps with batch 512, lr $3{\cdot}10^{-4}$ or $3{\cdot}10^{-3}$, and $\beta \in \{0.25, 1, 4\}$ where it applies. Each Blocksworld β-VAE setting has five seeds and the others three, giving 210 runs that we call the *main sweep*. Every run warms up the KL term linearly over the first 10% of training [16]. In a single-seed pilot without this warm-up the Blocksworld β-VAEs at β = 1 kept fewer units, among our three networks clearly so only at 2×256 (10 instead of 15 at the lower learning rate), where code10 and prod5 did not. The warm-up is therefore itself a training choice that can move the count, as Sønderby et al. also found [17].

4.2 Metrics

Besides active units we measure which facts the units detect and whether samples are legal.

- **Fact detectors.** For each unit and fact we fit a signed threshold on the posterior mean on development data and score it by F1 on the test split, adapting [18]. A unit with F1 ≥ 0.9 on some fact is a *detector*, and it can detect at most two facts that exclude each other, one at each end of its range.
- **Legality.** A decoded state is *well formed* when every block has exactly one place, and legal when, in addition, the constraints between blocks hold and clear and hand_empty agree with the positions.

We decode 20,000 samples from the prior, and as many from the aggregate posterior of the test states, by thresholding the outputs at 0.5. As a baseline we draw each block independently from its training distribution over positions, which gives 54% legal states. Since this baseline derives clear and hand_empty correctly, we score the VAE samples both *as decoded* and *on equal terms*, with these facts recomputed from the decoded positions.

4.3 Protocol

Before the main sweep we wrote down what we expected, and after a pilot we changed a few things, listed in Appendix A. We only compare models that reconstruct the data, at least 95% of the states exactly (with a few exceptions in Appendix A). We keep the 22 runs that did not converge, whose objective still moved by more than 2% in the last fifth of training. With so few seeds we compare their ranges instead of using statistical tests, and the analyses we chose after the main results, such as the rate, are descriptive.

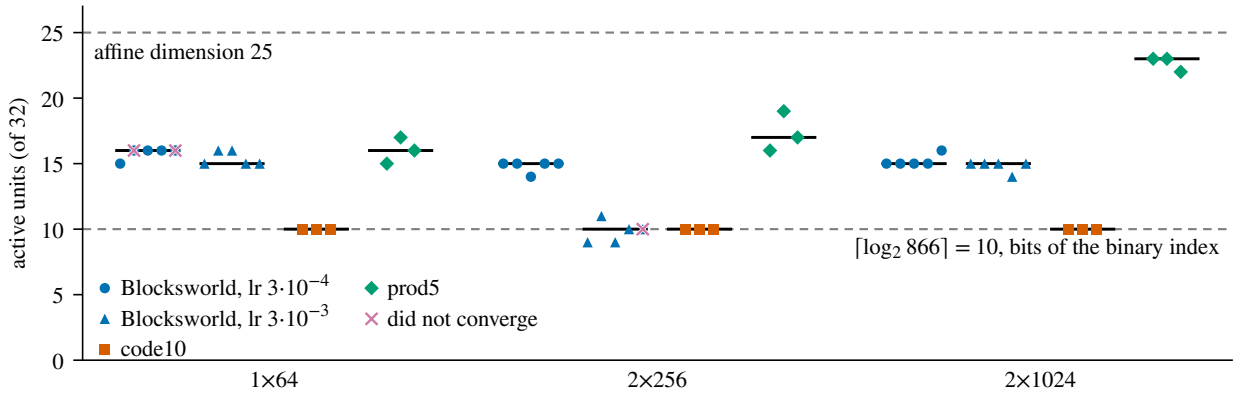


Figure 2. Active units (of 32) on raw states at β = 1, per seed and network. Blocksworld has five seeds per learning rate, code10 and prod5 three seeds at lr $3{\cdot}10^{-4}$. Black bars are medians, and crosses mark runs that did not converge.

5 Results

Unless stated otherwise, results are on the 36 facts at β = 1, and counts are medians over seeds on the test split.

5.1 Number of active units (Q2)

Like earlier VAEs with more latents than they need [2], [3], our β-VAEs leave many latents inactive (Figure 2). Five of the six Blocksworld settings keep 15 or 16 of their 32 latents active, which matches none of the dimensions of Table 1, and their seeds differ by at most two units. The sixth, the 2×256 network at the higher learning rate, keeps 10, which happens to be the number of bits of the binary index, but we did not test whether its units form such an index. The gap between active and inactive latents is clear in almost every run (Figure 6 in Appendix A), and the plain autoencoder keeps all 32 latents active, as it should.

code10 keeps 10 units in every run, as all three hypotheses expect, so on its own it cannot tell them apart. At the lower learning rate Blocksworld keeps five or six units more than code10, so the information hypothesis fails, and no Blocksworld run at this learning rate keeps fewer than code10, so the factor-count hypothesis fails too. The affine hypothesis holds at the two smaller networks, only just at 2×256, and only as an ordering, since no Blocksworld run comes near 25 units. prod5 grows with the network while Blocksworld does not, so this might just reflect capacity.

prod5, however, has constant bits where Blocksworld has derived facts. Once we drop these facts (Section 5.3), Blocksworld keeps 11 units at 2×256 and the lower learning rate, close to code10 (10) and far from prod5 (17). This is the pattern of the information hypothesis, although we saw it only after the main results and at one network. As expected, the 36 independent bits keep no active unit.

We also looked at the rate. It splits into the information the code carries about the state and the KL divergence between the aggregate posterior and the prior [19], which we call the *prior mismatch*. Since the information cannot exceed the entropy of the states, the rate above the entropy is a lower bound on this mismatch, and on Blocksworld it is at least 1.2 nats in every run. The rate stays near 8 nats while the count goes from 9 to 16 (Table 2), so the count is basically how many units share the same total KL, about half a nat per unit on average in the 15-unit settings. This does not explain why it settles there.

5.2 Learning rate and β (Q2)

As we expected, the count varies more between settings than between seeds, but only because of the 2×256 network at the higher learning rate. Asperti and Trentin also saw the count move with the learning rate [4, p.

Table 2. Active units (median and range over seeds, one value when all seeds agree), median rate and the median over seeds of the within-run mean KL per active unit, in nats, on raw states at β = 1 with seeds as in Figure 2. The entropy of the states is 6.47 nats for Blocksworld, 6.93 for code10 and 8.96 for prod5.

	1×64			2×256			2×1024		
	Active	**Rate**	**KL/unit**	**Active**	**Rate**	**KL/unit**	**Active**	**Rate**	**KL/unit**
Blocksworld, lr $3{\cdot}10^{-4}$	16 (15–16)	8.14	0.51	15 (14–15)	7.84	0.52	15 (15–16)	7.76	0.52
Blocksworld, lr $3{\cdot}10^{-3}$	15 (15–16)	8.13	0.54	10 (9–11)	7.92	0.79	15 (14–15)	7.90	0.53
code10	10	8.52	0.85	10	8.63	0.86	10	8.56	0.86
prod5	16 (15–17)	8.14	0.51	17 (16–19)	8.47	0.50	23 (22–23)	8.33	0.36

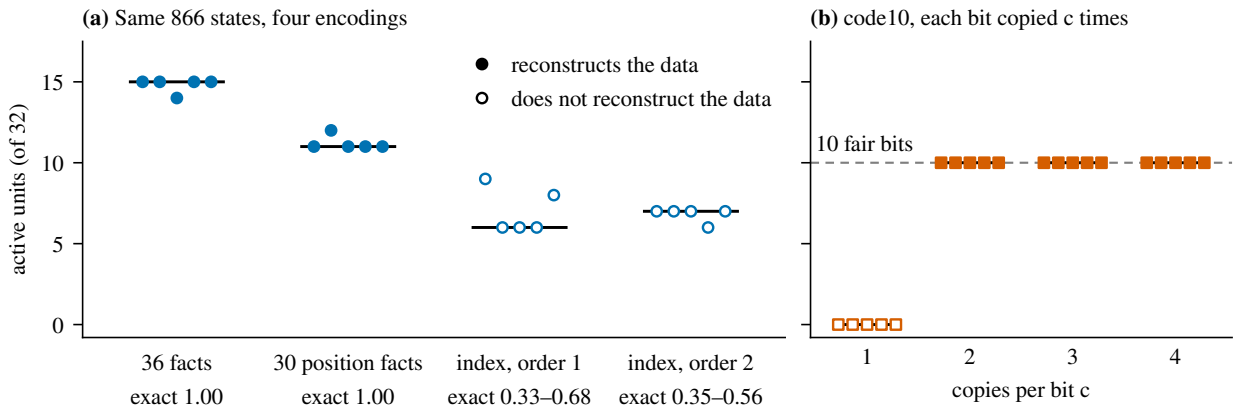


Figure 3. Active units (of 32) per seed in the follow-up, 2×256 at β = 1. (a) Four encodings of the 866 states, with the range of the share of test states reconstructed exactly. (b) code10 with each bit copied c times. Bars are medians, and hollow markers are runs that do not reconstruct the data by the rule of Section 4.3.

199446], in a CIFAR-10 comparison with no reported seed spread, a different criterion and learning rates below ours. There a higher learning rate left more units active, unlike in our 2×256 network, so only the sensitivity compares. This 10-unit network even fits the data better than the reference one, with a lower negative ELBO, so for this network the 15-unit codes are not the best ones.

Lowering β to 0.25 changes the medians by at most one unit, although Asperti notes that a smaller KL weight reduces the inactive variables [10, p. 6]. At β = 4 every Blocksworld run collapses, with no active unit at all.

5.3 Encoding and number of latents (Q2)

To see whether the count follows the encoding and the number of latents, we planned a follow-up after the main results, again writing down our expectations first. It keeps the reference configuration with five seeds and changes one of the following.

- The encoding, with only the 30 position facts or with the binary index in two random orders.
- The number of latents, from 8 to 64.
- The data, with code10 whose bits are copied one to four times.

Dropping the derived facts, which add no information, lowers the median from 15 to 11 (Figure 3a). The binary index keeps 6 and 7 units in its two orders, more than the 3 we had expected, but these models reconstruct only a third to two thirds of the states exactly, so they do not show how many units the full information needs.

Copies decide whether code10 uses its latents at all (Figure 3b). With each bit present once every seed keeps no active unit, like a decoder that ignores the code, and with two to four copies every seed keeps 10, as we had predicted.

From 24 latents on the median stays at 15 (Table 3b), as we expected and as Burda et al. found when adding dimensions [2, p. 8]. With 8, 10 or 12 latents every latent is active and every state is still reconstructed, so 8 latents are enough. The best median objective is at 8 latents, but 10 and 12 do worse than 32 and the seeds at 8 and 32 overlap. Together with Section 5.2, the objective in this network does not favour 15 units over 8 to 10, and training simply settles on the plateau.

5.4 Transformer activations (Q2)

On the transformer activations with learned γ the count grows with width, from 14 at T64 to 29 at T256 (Table 6 in Appendix A), but we have one transformer at each extreme, and width changes the size of the input

Table 3. Follow-up, five seeds per condition. Active units as median and range, then median rate and median negative ELBO, in nats. Column 32 of (b) is the reference configuration.

	(a) Encoding, L = 32			(b) Latents L, 36 facts						
	30 facts	index 1	index 2	8	10	12	16	24	32	64
Active units	11	6	7	8	10	12	14	15	15	15
range	11–12	6–9	6–7	8	10	12	13–15	15–16	14–15	15–16
Rate	6.49	2.12	2.07	7.95	7.94	7.93	7.92	7.87	7.84	7.89
Negative ELBO	8.44	6.70	6.71	9.30	9.53	9.63	9.42	9.59	9.45	9.62

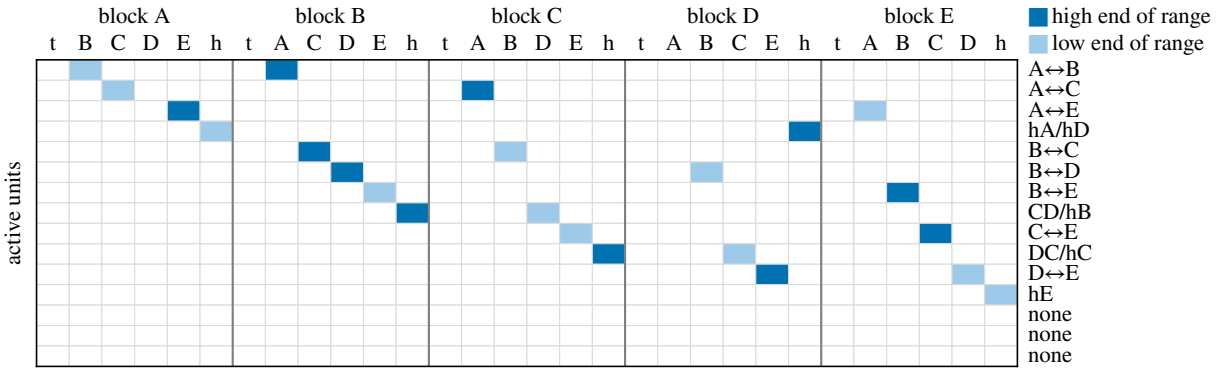


Figure 4. Facts detected (F1 ≥ 0.9) by the 15 active units (rows) of the reference configuration, seed 1, the seed with the median number of two-fact units, and the last three detect none. Columns are the 30 position facts, six per block X, with t for on_table(X), Y for on(X, Y) and h for holding(X). In the row labels X↔Y is a reciprocal unit, XY is on(X, Y), hX is holding(X) and a slash joins the two facts of any other unit.

too. Part of the growth might come from the objective, which sums the reconstruction over more coordinates against the same KL, and this is why Asperti and Trentin scale the KL by four when the pixels grow four times [4, p. 199445].

5.5 What the units encode (Q3)

The active units do not match the facts one to one. We expected few units to detect a fact, but in the reference configuration most of them do, and most have one fact at each end of their range. The first row of Figure 4 is typical, as that unit is low when A is on B and high when B is on A. In 44 of the 74 active units over five seeds the two facts are on(X, Y) and on(Y, X), and we call these units *reciprocal units*.

The detector does not force this pairing, since many other pairs of facts also exclude each other, for example two positions of the same block, which only one unit uses. We did not check whether reciprocal pairs just follow from how often the facts occur together, or whether these units help set the count. The 1×64 network at the lower learning rate keeps a similar count with at most three reciprocal units per run, so similar counts can come with very different codes (Table 7 in Appendix A). The top-k SAE with k = 5, which we chose to match the one vector per block, separates the blocks more cleanly than the β-VAE, but it uses about twice as many units. Contrary to what we expected, it does not detect more facts than the best β-VAE.

5.6 Legality of samples (Q4)

In the main sweep the β-VAEs at β = 0.25 and 1 reconstruct every state from their posterior means, but their samples are often illegal (Figure 5). Since at β = 1 our pilot models already missed the legality level we had planned to test, here we only describe what we see. As decoded, the prior samples of most runs are less often legal than the per-block baseline. On equal terms, at β = 1, the two larger networks beat it in every run, and the smallest one never does.

Samples from the aggregate posterior are better, but at β = 1 none reaches 95% legal states as decoded, and at β = 0.25 the two larger networks do in almost every run. Most illegal samples have a block that is not in exactly one place or a wrong derived fact, as in Figure 1b, but among well-formed samples the constraints between blocks mostly hold (Table 4).

The gap between prior and aggregate-posterior samples fits the prior mismatch of Section 5.1, but we did not measure how much illegality it causes. At β = 1 part of the failure of the aggregate-posterior samples might come from the noise of the posterior itself, since the posterior means of the same states decode correctly.

6 Conclusions and limitations

The count (Q1, Q2). For the Blocksworld states there is no single right dimension, and none of the ones we fixed before training predicted the count across settings. On the 36 facts at β = 1 the rate stays near 8 nats while the count moves, so the count is just how many units share it, which does not tell us why it settles where it does. The learning rate in one network, the encoding and, in a single-seed pilot, the warm-up move the count, the last in line with Sønderby et al. [17]. Dropping the derived facts brings the count close to code10, as the information hypothesis predicted, although we saw this only after the main results and at one network.

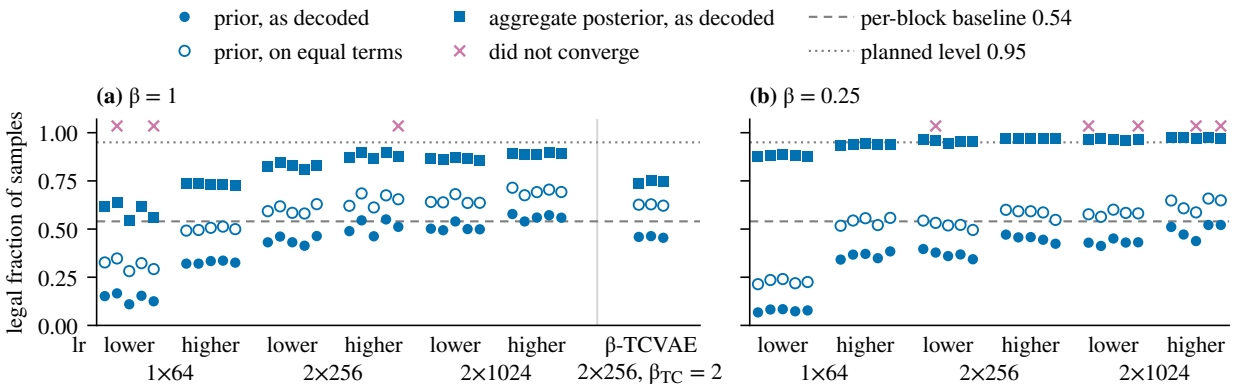


Figure 5. Legality per run of samples thresholded at 0.5, for prior samples as decoded and on equal terms and for aggregate-posterior samples as decoded. (a) β-VAEs on raw states at $\beta = 1$ (five seeds) and the β-TCVAE at $\beta_{TC} = 2$ (three seeds). (b) β-VAEs at $\beta = 0.25$. The lower lr is $3{\cdot}10^{-4}$ and the higher $3{\cdot}10^{-3}$, and a cross above a run marks that it did not converge.

Table 4. Where samples fail, for the β-VAEs on raw states at $\beta = 1$. Each cell is the range of the six configuration medians, and ≥ marks the smallest value over runs. Row 2 is computed among well-formed samples, and in parentheses for blocks drawn independently from the model's marginals.

Fraction of samples	Prior	Aggregate posterior
well formed	0.36–0.72	0.76–0.94
constraints between blocks hold (independent blocks)	0.87–0.97 (0.60–0.72)	≥ 0.96 (0.53–0.56)
clear and hand_empty consistent, given legal positions	0.46–0.81	0.82–0.97

Copied bits get informative codes, as the reconstruction–KL balance suggests, but the balance does not say how many units these codes use, and in the 2×256 network the objective does not prefer 15 units over 8 to 10. In the reference settings the median count stays at 15 as the latents grow from 24 to 64, as Burda et al. found [2], which fits Asperti's suggestion of an internal dimension for a given architecture [3, p. 1]. However, as Asperti and Trentin also found [4], the count moves with training choices, so we read it as a property of the states together with the network, the encoding and the training.

The units (Q3). In the reference configuration most active units detect one or two facts, mostly a block on another and the reverse at opposite ends of their range, so they are tied to facts but not one to one, and similar counts occur with very different codes.

Sampling (Q4). The β-VAEs of the main sweep at $\beta = 0.25$ and 1 reconstruct every state from their posterior means, but their prior samples are often illegal, as the samples of Asperti's VAE trained on images of one to three tiles often had more tiles [3, pp. 5–6]. The gap to aggregate-posterior samples fits the prior mismatch, which Asperti indicates as the problem when reconstruction is good but generation is bad [10, p. 7], but we did not test it. As a next experiment we would keep the trained decoder and fit a Gaussian mixture to the posterior means of the training states [20], [21], or a second VAE as in the two-stage VAE of [7], which he also suggests for this case. We would then check whether its samples are more often legal than those from the prior.

Limitations. Our results hold for the small networks, data and encodings we used, and they come from coarse comparisons over a few seeds. The follow-up used one network and learning rate, and on activations we had one transformer at each extreme of width. Finally, the analyses we chose after the main results reuse test data we had already seen.

Use of AI assistance

The author designed the project and its experiments. The code was written with AI coding assistants (Anthropic's Claude and OpenAI Codex), which also helped to revise the text. The author checked the results and is responsible for the report.

References

[1] J. Slaney and S. Thiébaux, "Blocks World Revisited," *Artificial Intelligence*, vol. 125, no. 1–2, pp. 119–153, 2001, doi: 10.1016/S0004-3702(00)00079-5.

[2] Y. Burda, R. Grosse, and R. Salakhutdinov, "Importance Weighted Autoencoders," in *International Conference on Learning Representations (ICLR)*, 2016.

[3] A. Asperti, "Sparsity in Variational Autoencoders," *arXiv preprint arXiv:1812.07238v3*, 2018, [Online]. Available: https://arxiv.org/abs/1812.07238

[4] A. Asperti and M. Trentin, "Balancing Reconstruction Error and Kullback-Leibler Divergence in Variational Autoencoders," *IEEE*

Access, vol. 8, pp. 199440–199448, 2020, doi: 10.1109/ACCESS.2020.3034828.
[5] I. Higgins *et al.*, "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," in *International Conference on Learning Representations (ICLR)*, 2017.
[6] F. Locatello *et al.*, "Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 4114–4124.
[7] B. Dai and D. Wipf, "Diagnosing and Enhancing VAE Models," in *International Conference on Learning Representations (ICLR)*, 2019.
[8] Y. Zheng, T. He, Y. Qiu, and D. Wipf, "Learning Manifold Dimensions with Conditional Variational Autoencoders," in *Advances in Neural Information Processing Systems*, 2022, pp. 34709–34721.
[9] A. A. Alemi, B. Poole, I. Fischer, J. V. Dillon, R. A. Saurous, and K. Murphy, "Fixing a Broken ELBO," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 159–168.
[10] A. Asperti, "Variational Autoencoders and the Variable Collapse Phenomenon," *Sensors & Transducers*, vol. 234, no. 6, pp. 1–8, 2019, [Online]. Available: https://www.sensorsportal.com/HTML/DIGEST/june_2019/Vol_234/P_3085.pdf
[11] X. Chen *et al.*, "Variational Lossy Autoencoder," in *International Conference on Learning Representations (ICLR)*, 2017.
[12] A. Makhzani and B. J. Frey, "k-Sparse Autoencoders," in *International Conference on Learning Representations (ICLR)*, 2014.
[13] R. T. Q. Chen, X. Li, R. Grosse, and D. Duvenaud, "Isolating Sources of Disentanglement in Variational Autoencoders," in *Advances in Neural Information Processing Systems*, 2018.
[14] E. Jang, S. Gu, and B. Poole, "Categorical Reparameterization with Gumbel-Softmax," in *International Conference on Learning Representations (ICLR)*, 2017.
[15] D. P. Kingma and J. L. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, 2015.
[16] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating Sentences from a Continuous Space," in *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, 2016, pp. 10–21. doi: 10.18653/v1/K16-1002.
[17] C. K. Sønderby, T. Raiko, L. Maaløe, S. K. Sønderby, and O. Winther, "Ladder Variational Autoencoders," in *Advances in Neural Information Processing Systems*, 2016.
[18] A. Karvonen *et al.*, "Measuring Progress in Dictionary Learning for Language Model Interpretability with Board Game Models," in *Advances in Neural Information Processing Systems*, 2024, pp. 83091–83118.
[19] M. D. Hoffman and M. J. Johnson, "ELBO Surgery: Yet Another Way to Carve up the Variational Evidence Lower Bound," in *NIPS 2016 Workshop on Advances in Approximate Bayesian Inference*, Barcelona, Spain, 2016. [Online]. Available: http://approximateinference.org/2016/accepted/HoffmanJohnson2016.pdf
[20] P. Ghosh, M. S. M. Sajjadi, A. Vergari, M. Black, and B. Schölkopf, "From Variational to Deterministic Autoencoders," in *International Conference on Learning Representations (ICLR)*, 2020.
[21] A. Asperti, D. Evangelista, and E. Loli Piccolomini, "A Survey on Variational Autoencoders from a Green AI Perspective," *SN Computer Science*, vol. 2, no. 4, 2021, doi: 10.1007/s42979-021-00702-9.
[22] C. Eastwood and C. K. I. Williams, "A Framework for the Quantitative Evaluation of Disentangled Representations," in *International Conference on Learning Representations (ICLR)*, 2018.

Appendix A: Planned predictions, changes to the plan and code map

Planned predictions. Table 5 lists the main predictions we wrote down before the main sweep or the follow-up, with their outcomes, and the experiment READMEs list the secondary ones. Counts are medians at $\beta = 1$ on raw states unless stated. Models that did not reconstruct the data (Section 4.3) entered no prediction, except those for the index, the copies and β, which use every run.

Table 5. Main planned predictions and their outcomes. The Section 3.2 hypotheses were decided per network at the lower learning rate.

Prediction	Outcome
Fewer than 25 active units on Blocksworld	holds (at most 16)
Information hypothesis at every network (Section 3.2)	fails at all three
Affine hypothesis (decision rule only, no predicted outcome)	holds at 1×64, barely at 2×256
Factor-count hypothesis (decision rule only, no predicted outcome)	holds at no network
Spread of configuration medians at least 3 and above the seed spread	holds, from one configuration
Learned γ adds ≥ 2 units on state-projected activations (Table 6), ranges separated, at every transformer	fails at all five
$k = 5$ SAE has detectors for ≥ 24 of the 30 position facts, more than every Gaussian-latent model	fails (24 against 25)
Prior samples beat the per-block baseline, legality falls with the count	not tested (pilot below 95%)
Index keeps median ≤ 3, no run > 5, ≥ 10 fewer than 36 facts (follow-up)	fails on all three
code10 keeps ≤ 1 unit with one copy, 10 (runs 9–11) with 2–4 (follow-up)	holds
Medians at 24, 32 and 64 latents within two units (follow-up)	holds

Changes to the plan. After the pilots we changed five things before looking at the main sweep. We dropped a 4×256 network, which could not reconstruct the data even with a KL warm-up, and adapted the predictions from four networks to three. We added the KL warm-up to every run, as the plan prescribed in this case, and trained for 60,000 steps instead of 30,000, since the pilot as a whole had not converged, with an extra convergence rule for the models whose objective is near zero and one for a learned γ. We left out comparison models that never reconstruct the data, and we kept the categorical VAE, whose objective still moved by 4.0% in its pilot, a case the plan did not cover, although strictly the plan would have stopped the sweep.

Other notes. Leaving out the 22 runs that did not converge changes no primary outcome, and of the secondary ones only the one on β. The follow-up ran as planned. On 30 October 2026 we added the KL per active unit and legality on equal terms from stored outputs only. Separately written re-implementations of the scoring reproduced every count and outcome, but they were written by models of the same family as those that ran the analyses, so they are not external replications.

Additional figure and tables. Figure 6 shows the gap between active and inactive latents of Section 5.1, Table 6 the counts on transformer activations of Section 5.4 and Table 7 the alignment of units with facts of Section 5.5.

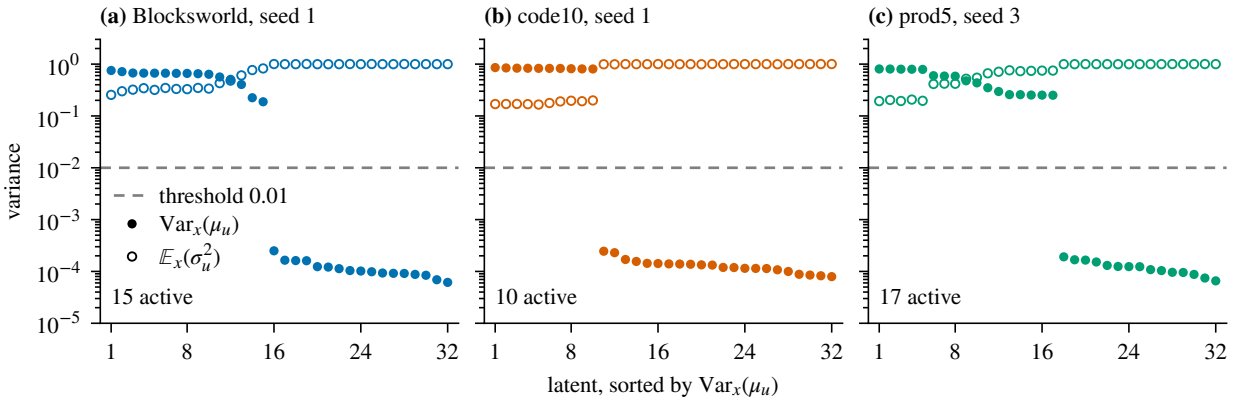


Figure 6. Variance of the posterior mean (filled) and mean posterior variance (hollow) of each latent, sorted by the former, over the test states (all states, equally weighted, for code10 and prod5). Each panel shows the seed with the median count at 2×256, β = 1 and lr $3{\cdot}10^{-4}$, and (a) is the reference configuration.

Table 6. Active units on transformer activations, median (min–max) over three seeds, for the 2×256 β-VAE at β = 1 and lr $3{\cdot}10^{-4}$. K_{probe} is the fewest principal components with which linear probes come within 0.5 percentage points of their accuracy on all coordinates, and $K_{99\%}$ the fewest that explain 99% of the variance. Projected stands for the state-projected activations, the least-squares prediction of the activations from the current facts (rank 25). The last two rows vary β on T128a, where no run converges at β = 0.25.

	Comparison counts		Active units (of 64)		
Transformer	**K_{probe}**	**$K_{99\%}$**	**Projected, γ = 1**	**Projected, learned γ**	**Full, learned γ**
T64	25	39	14 (13–14)	14 (13–14)	14 (14–16)
T128a	26	88	14 (14–16)	17 (16–17)	20 (20–21)
T128b	27	91	17 (16–17)	16 (15–17)	22 (21–22)
T128c	27	93	16 (13–18)	18 (17–18)	22 (20–23)
T256	31	208	19 (17–19)	21 (18–22)	29 (28–30)
T128a, β = 0.25					62 (61–63)
T128a, β = 4					14 (13–16)

Table 7. Fact alignment on raw states at β = 1 and lr $3{\cdot}10^{-4}$ unless stated. Coverage is the number of the 30 position facts with a detector among the active units (for the SAE, the alive units), per seed for the models compared in the text and a range otherwise, and 25 is a natural level since a code can leave one position per block implicit. The detector fraction is the share of active units (for the SAE, of alive units, firing on at least 0.5% of test states) that detect a fact. DCI is the disentanglement score of [22], which measures how much each unit is used for one block variable only.

Model (raw states)	**Coverage (of 30)**	**Detector fraction**	**DCI**	**Reciprocal units**
top-k SAE, k = 5	24, 25, 25	0.87–0.91	0.96–1.00	n/a
top-k SAE, k = 10	25, 25, 26	0.71–0.93	0.95–1.00	n/a
β-TCVAE, β_{TC} = 2, 2×256	20–22	0.81–0.94	0.58–0.65	n/a
β-VAE, reference configuration	23, 25, 25, 23, 25	0.80–1.00	0.55–0.65	8–9
β-VAE, 1×64	3–10	0.12–0.44	0.27–0.38	0–3
β-VAE, 1×64, lr $3{\cdot}10^{-3}$	25	0.93–1.00	0.60–0.69	9–10
β-VAE, 2×256, lr $3{\cdot}10^{-3}$	6–12	0.66–0.91	0.39–0.54	0–2
β-VAE, 2×1024	24–25	0.86–0.94	0.56–0.61	9–10
β-VAE, 2×1024, lr $3{\cdot}10^{-3}$	14–24	0.60–0.94	0.51–0.63	5–10
β = 4, β-TCVAE at β_{TC} = 6, categorical VAE	no reconstruction (all β-VAEs at β = 4), not scored			

Code map. Each directory under `experiments/` holds its plan, a `README.md` with its status and its scripts, with JSON results where it ran. The shared code is in `src/pwlib/`, and `pw02–pw04` hold the plans replaced after the pilots by `pw02a–pw04a`. `report/make_figures.py` draws the figures and writes the numbers behind them to `report/figures/tables.md`, with their sources.

Report	**Directory**	**Report**	**Directory**
Section 3.1	`pw01_state_space_ground_truth`	5.5, alignment	`pw03a_fact_alignment_amended`
counts of 5.1, 5.2, 5.4	`pw02a_active_units_amended`	5.5, post-hoc analyses	`pw05_unit_anatomy`
5.3, follow-up	`pw06_encoding_and_latent_size`	5.6, legality	`pw04a_sample_legality_amended`
KL per unit, equal terms	`pw07_unit_kl_and_matched_legality`	full chronology	`docs/EXPERIMENT_LOG.md`